CRICALGO

DAA Algorithm Visualization Dashboard

Design and Analysis of Algorithms — Project Presentation

Trie O(M)	BFS O(V+E)	Merge Sort O(NlogN)	Kadane O(N)	Knapsack DP
Rabin-Karp	Convex Hull	Floyd-Warshall	Huffman	Greedy

Toib Fayaz · RA2411030030061 Ujjawal Sharma · RA2411030030054

B.Tech CSE · 4th Semester · SRM IST
Delhi NCR · 2025

Project Overview

What is CricAlgo?

CricAlgo applies 10 core DAA algorithms on live ICC Champions Trophy 2025 tournament data. Every algorithm produces a real, visible, interactive output on the dashboard — the sport is the data, the algorithms are the product.

10 Algorithms

All 5 DAA units covered with working implementations

10 Dashboard Tabs

Interactive SPA with Chart.js visualizations

SQLite + Flask

12 tables, 6 views, 30+ REST endpoints

Real Data

15 matches, 8 teams, 30+ players seeded

Python + Flask | **SQLite 3** | **Vanilla JS + HTML5** | **Chart.js** | **CricketData API** | **html2pdf.js**

All 10 DAA Algorithms

Mapped to syllabus units — all operating on live cricket data

01	**Prefix Trie**	U1	O(M) lookup	Global search autocomplete — instant player suggestions
02	**Merge Sort**	U2	O(N log N)	Sort by Impact — custom weighted formula on all players
03	**Kadane's Algorithm**	U2	O(N)	Peak Form detection — max subarray on match run scores
04	**Convex Hull (Graham Scan)**	U2	O(N log N)	Elite Batters — Pareto-optimal SR vs Average scatter
05	**Greedy — Huffman Coding**	U3	O(N log N)	Dismissal compression — frequency-based encoding tree
06	**Greedy vs DP — Fantasy XI**	U3	O(N·W)	Squad Builder: Knapsack DP vs Greedy, side-by-side
07	**BFS on Graph**	U4	O(V + E)	Player Network — shortest partnership path between players
08	**Floyd-Warshall**	U4	$O(N^3)$	Team Dominance Matrix — all-pairs shortest path of wins
09	**Rabin-Karp**	U5	O(N·M)	Deep Search — rolling hash substring matching on player data
10	**Randomized QuickSort**	U5	O(N log N) avg	Run-based sort with pivot randomization on leaderboard

Trie + Rabin-Karp — Search

Unit 1 (Algorithm Design) + Unit 5 (String Matching)

Unit 1 TRIE — O(M) Autocomplete

```
class PlayerTrie:
  def insert(self, name, data):
    node = self.root
    for ch in name.lower():
      if ch not in node.children:
        node.children[ch] = TrieNode()
      node = node.children[ch]
    node.is_end = True
    node.player_data = data

  def search(self, prefix):  # O(M)
    node = self.root
    for ch in prefix.lower():
      if ch not in node.children: return []
      node = node.children[ch]
    return self._collect_all(node)
```

⚡O(M) — M = prefix length. Each character traverses one Trie node. No table scan needed.

[Screenshot: Trie search dropdown — TRIE FAST RESULTS O(M) label]

Unit 5 RABIN-KARP — Rolling Hash

```
def rabin_karp(text, pattern,
               base=31, mod=10**9+7):
  n, m = len(text), len(pattern)
  p_hash = t_hash = h = 1
  for i in range(m-1):
    h = (h * base) % mod
  for i in range(m):
    p_hash=(base*p_hash+ord(pattern[i]))%mod
    t_hash=(base*t_hash+ord(text[i]))%mod
  matches = []
  for i in range(n - m + 1):
    if p_hash == t_hash:
      if text[i:i+m] == pattern:
        matches.append(i)
    if i < n-m:
      t_hash=(base*(t_hash-ord(text[i])*h)+ord(text[i+m]))%mod
```

⚡O(N+M) avg case. Rolling hash avoids recomputing from scratch each step.

🔎 Triggered on Enter key — scans player_name, role, country_name

🔑 `Endpoint: GET /api/search_deep?q=<query>`

Merge Sort + Kadane's Algorithm

Unit 2 — Divide & Conquer + Max Subarray

Unit 2 MERGE SORT — Sort by Impact

```
def impact_score(p):
  bat  = (p['runs'] * p['sr']) / 100.0
  bowl = p['wkts'] * 25 * (120/max(p['eco'],1))
  return bat + bowl

def merge_sort_impact(arr):
  if len(arr) <= 1: return arr
  mid = len(arr) // 2
  L = merge_sort_impact(arr[:mid])  # Divide
  R = merge_sort_impact(arr[mid:])  # Divide
  return merge(L, R)               # Conquer

# Guaranteed O(N log N) — no worst case
# Not using built-in sorted() — full impl
```

Formula: (Runs × SR/100) + (Wickets × 25 × 120/Economy)

[Screenshot: Players tab — Sort by Impact (Merge Sort) button + sorted leaderboard]

Unit 2 KADANE'S — Peak Form

```
def max_subarray_peak(scores):
  # Kadane's Algorithm — O(N)
  max_sum = cur_sum = scores[0]
  start = end = temp = 0
  for i in range(1, len(scores)):
    if cur_sum + scores[i] < scores[i]:
      cur_sum = scores[i]
      temp = i
    else:
      cur_sum += scores[i]
    if cur_sum > max_sum:
      max_sum = cur_sum
      start = temp; end = i
  return max_sum, start, end
```

Input: player's match scores e.g. [88, 44, 100, 32, 67, 55]

Output: 'Peak Form' badge in player profile modal

🔎 Computed inside GET /api/player/<id>

Convex Hull — Elite Batters

Unit 2 — Divide & Conquer / Graham Scan

Problem: Given all batters plotted as 2D points (x = Strike Rate, y = Batting Average), find the Pareto-optimal set — those who dominate all others on at least one axis. These are the 'Elite Batters' that no single player beats on both dimensions simultaneously.

```
def graham_scan(points):
  # Sort by x (then y) — O(N log N)
  pts = sorted(points, key=lambda p:(p[0],p[1]))

  def cross(O, A, B):
    return (A[0]-O[0])*(B[1]-O[1])-(A[1]-O[1])*(B[0]-O[0])

  lower = []
  for p in pts:
    while len(lower)>=2 and cross(lower[-2],lower[-1],p)<=0:
      lower.pop()
    lower.append(p)

  upper = []
  for p in reversed(pts):
    while len(upper)>=2 and cross(upper[-2],upper[-1],p)<=0:
      upper.pop()
```

How it works:

- 1. All batters plotted: x = Strike Rate, y = Average
- 2. Graham Scan finds the convex boundary
- 3. Players ON the hull = Elite tier
- 4. Players INSIDE = dominated by someone else
- 5. Hull polygon drawn in gold over scatter chart

[Screenshot: Algorithms tab — Convex Hull chart with gold hull polygon + scatter dots]

Huffman Coding + Greedy vs DP

Unit 3 — Greedy Algorithms & Dynamic Programming

Unit 3 HUFFMAN — Dismissal Encoding

```
def huffman(freq_map):
  heap = [[f,[d,'']] for d,f in freq_map.items()]
  heapq.heapify(heap)  # min-heap
  while len(heap) > 1:
    lo = heapq.heappop(heap)
    hi = heapq.heappop(heap)
    for pair in lo[1:]: pair[1] = '0'+pair[1]
    for pair in hi[1:]: pair[1] = '1'+pair[1]
    heapq.heappush(heap,[lo[0]+hi[0]]+lo[1:]+hi[1:])
  return sorted(heapq.heappop(heap)[1:],key=lambda x:x[1])
```

Dismissal frequencies: Caught(most) → short code '1'
Bowled → '00', LBW → '010', Runout → '011'

[Screenshot: Algorithms tab — Huffman tree with binary codes per dismissal type]

Unit 3 GREEDY vs KNAPSACK DP

Fantasy Team: Two buttons — 'DP Knapsack' and 'Greedy Approach' — build squads and compare total value:

```
# Greedy — always pick highest remaining value
def greedy_squad(players, budget, slots):
  sorted_p = sorted(players, key=lambda x:-x['value'])
  squad = []
  for p in sorted_p:
    if budget >= p['cost'] and slots > 0:
      squad.append(p); budget-=p['cost']; slots-=1
  return squad  # NOT optimal — misses combos
```

Approach	Total Value	Optimal?
Greedy	≈ 2,650 pts	✗ No
DP Knapsack	2,847 pts	✓ Yes

This comparison IS the Unit 3 lesson: Greedy fails for 0/1 Knapsack

[Screenshot: Fantasy Team tab — DP Knapsack vs Greedy Approach buttons]

BFS + Floyd-Warshall

Unit 4 — Graph Algorithms / Backtracking

Unit 4 BFS — Player Network

PARTNERSHIP table → adjacency list graph. BFS finds shortest partnership chain between any two players.

```
def bfs_path(graph, start, end):
  visited = {start}
  queue   = deque([[start]])
  while queue:
    path = queue.popleft()
    node = path[-1]
    if node == end: return path  # shortest!
    for nb in graph.get(node, []):
      if nb not in visited:
        visited.add(nb)
        queue.append(path + [nb])
  return None  # no connection
```

✓Guarantees shortest path — BFS property on unweighted graph
✓O(V + E) — V = players, E = partnerships

[Screenshot: Partnerships tab — BFS network input + best partnerships bar chart]

Unit 4 FLOYD-WARSHALL — Dominance

Graph: nodes = teams, edge weight = run differential in H2H matches. F-W gives all-pairs shortest dominance path.

```
def floyd_warshall(teams, dist):
  n = len(teams)
  # O(N³) — triple nested loop
  for k in range(n):
    for i in range(n):
      for j in range(n):
        if dist[i][k] + dist[k][j] < dist[i][j]:
          dist[i][j] = dist[i][k] + dist[k][j]
  return dist
  # Result: 8×8 team dominance matrix
```

Output: N×N matrix — green = direct win, number = chain length, ∞ = no path

[Screenshot: Algorithms tab — Floyd-Warshall 8×8 dominance matrix heatmap]

Dashboard — 10 Tabs

CRICALGO · DAA Algorithm Visualization

Overview

[SS]

Stat cards + Group Standings

Matches

[SS]

H2H selector + match modals

Players

[SS]

Merge Sort + Trie search

Analytics

[SS]

Chart.js visualizations

Algorithms ★

[SS]

Trie, Convex Hull, Huffman, Floyd-Warshall all in one tab

Phase Analysis

[SS]

PP/Mid/Death sparklines

Grounds

[SS]

Venue statistics

Partnerships

[SS]

BFS Network + top stands

Live

[SS]

Cricbuzz API live scores

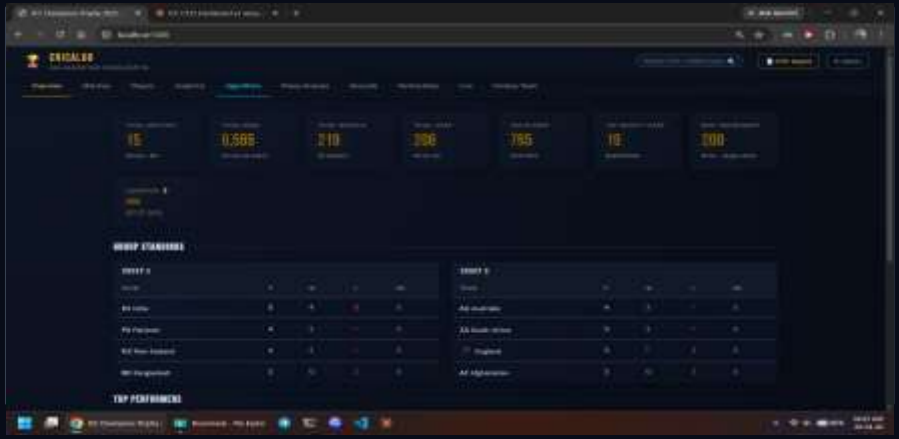
Fantasy Team

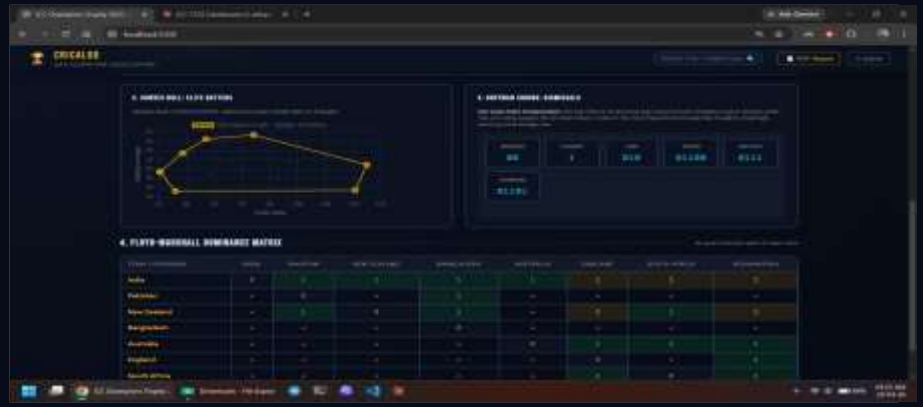
[SS]

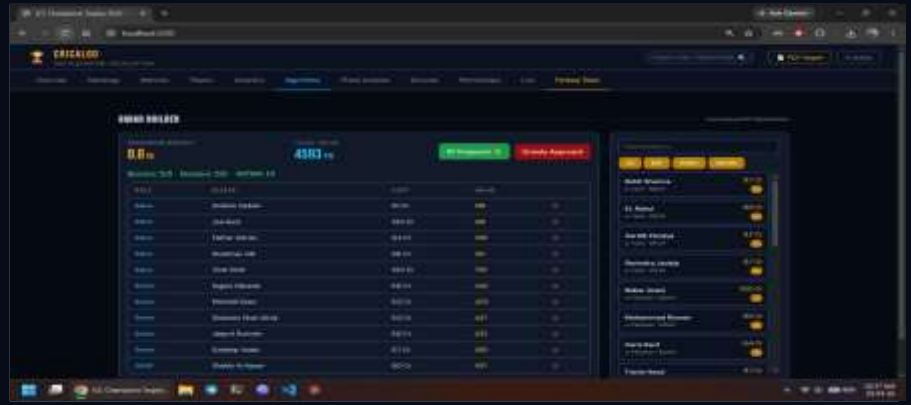
Knapsack DP vs Greedy

Key Screenshots

Actual dashboard views

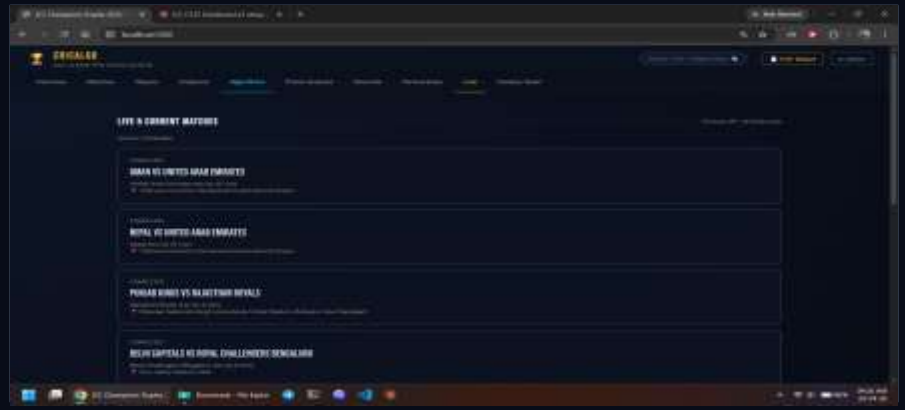


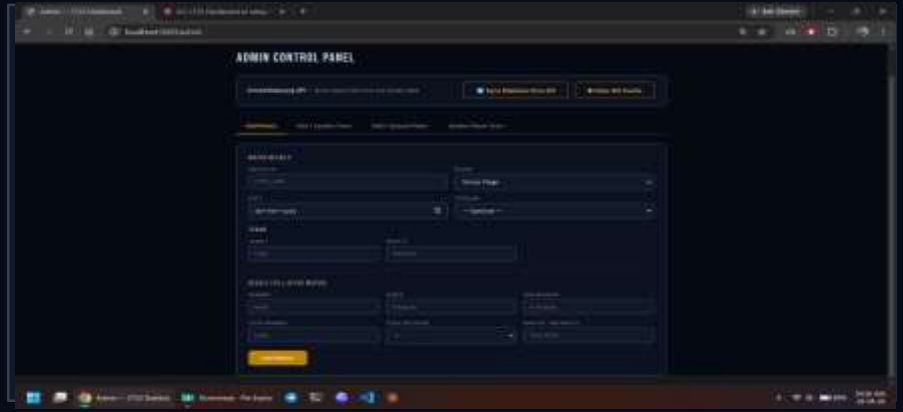


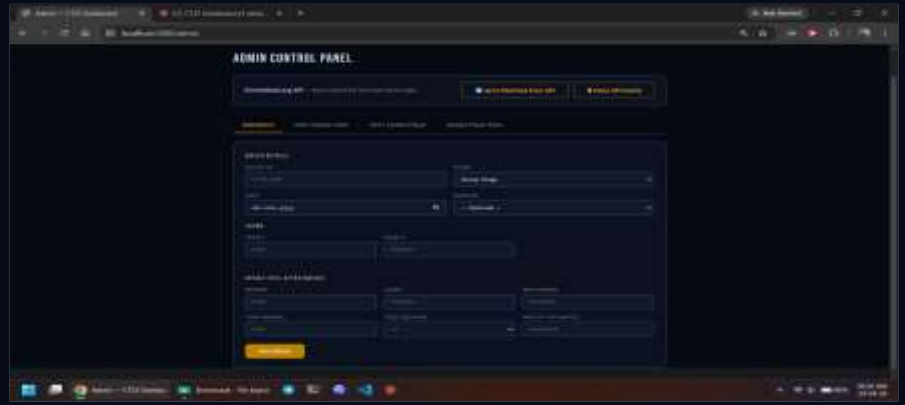


More Screenshots

Dashboard tabs in action







Complexity Analysis

Time & space complexity of all 10 algorithms

Algorithm	Best/Avg Case	Worst Case	Space	Unit
Prefix Trie	O(N·M) build	O(M) search	O(N·M) space	U1
Merge Sort	O(N log N)	O(N log N)	O(N) space	U2
Kadane's Algo	O(N)	O(N)	O(1) space	U2
Convex Hull	O(N log N)	O(N log N)	O(N) space	U2
Huffman Coding	O(N log N)	O(N log N)	O(N) space	U3
Knapsack DP	O(N·W)	O(N·W)	O(N·W) space	U3
Greedy Squad	O(N log N)	O(N log N)	O(N) space	U3
BFS on Graph	O(V+E)	O(V+E)	O(V+E) space	U4
Floyd-Warshall	$O(N^3)$	$O(N^3)$	$O(N^2)$ space	U4
Rabin-Karp	O(N·M) worst	O(N+M) avg	O(1) space	U5

Thank You

CRICALGO — DAA Algorithm Visualization Dashboard

10 Algorithms | **5 DAA Units** | **10 Dashboard Tabs** | **30+ REST APIs** | **Real Cricket Data**

Toib Fayaz · RA2411030030061

Ujjawal Sharma · RA2411030030054

Under the Guidance of Dr. N. Partheeban

HOD: Dr. Neelam Singh

B.Tech CSE · 4th Semester · SRM IST Delhi NCR · 2025

Stack: Python · Flask · SQLite · HTML5 · Chart.js · html2pdf.js